

PROYECTO INTEGRADOR DE MÉTODOS NUMÉRICOS

Fase 1: Comprensión del Artículo y Formulación del Problema Computacional

Familia Óptima de Métodos Iterativos Libres de Derivadas para la Búsqueda de Raíces Múltiples

Análisis teórico, articulación curricular y formulación algorítmica

Artículo Científico de Referencia:

Jerezano, J. H., Chicharro, F. I., & Garrido-Sáez, N. (2026). *Derivative-free optimal multiple-root finding family of iterative methods*. *Computational and Applied Mathematics*, 45:304. [doi:10.1007/s40314-026-03702-8](https://doi.org/10.1007/s40314-026-03702-8)

Integrantes del Equipo:

Ailyn Sofía Gómez Rodríguez

ailyn.gomez@estudiantes.uexternado.edu.co

Julián Santiago Jiménez

julian.jimenez01@estudiantes.uexternado.edu.co

Julián Esteban Duarte

julian.duarte@estudiantes.uexternado.edu.co

Docente Titular:

Edilberto Edwin Muñoz Chipatecua

Departamento de Matemáticas

Repositorio del Proyecto:

<https://github.com/ailynsof/Proyecto-M-todos-Numericos>

1. Comprensión del Artículo Científico

1.1. Problema Abordado e Intuición Teórica

En la computación científica y la Ciencia de Datos, gran parte de los problemas aplicados se formulan como la búsqueda de un valor α tal que una función continua g satisfaga $g(\alpha) = 0$. A este valor α se le denomina **raíz** o solución de la ecuación no lineal.

En diversos fenómenos físicos y termodinámicos —como el cálculo del volumen molar en la ecuación de Van der Waals o la densidad espectral en la ley de radiación de Planck— las soluciones presentan **multiplicidad** $m \geq 2$. Matemáticamente, α es una raíz de multiplicidad m si la función y sus primeras $m - 1$ derivadas se anulan en α , mientras que su m -ésima derivada es no nula:

$$g(\alpha) = g'(\alpha) = \dots = g^{(m-1)}(\alpha) = 0, \quad \text{con } g^{(m)}(\alpha) \neq 0. \quad (1)$$

¿Por qué las raíces múltiples degradan los métodos convencionales?

- **Aplanamiento geométrico:** Mientras una raíz simple ($m = 1$) corta el eje x con pendiente pronunciada, en una raíz múltiple la curva se vuelve tangente y horizontal al eje, asemejando un valle muy plano.
- **Frenado del método de Newton estándar:** En el método clásico de Newton-Raphson ($x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)}$), tanto el residuo $g(x_k)$ como la pendiente tangente $g'(x_k)$ se aproximan a cero simultáneamente. La división entre magnitudes infinitesimales reduce los pasos drásticamente, frenando la convergencia de rapidez cuadrática ($p = 2$) a una lenta marcha lineal ($p = 1$).
- **Restauración con Newton modificado:** Para compensar el aplanamiento, Ernst Schröder propuso amplificar el paso multiplicándolo por la multiplicidad m : $x_{k+1} = x_k - m \frac{g(x_k)}{g'(x_k)}$. Esto restaura con éxito el orden cuadrático ($p = 2$).

El desafío computacional: Prescindir de derivadas analíticas. El método de Newton modificado exige evaluar la derivada analítica $g'(x_k)$ en cada paso. En problemas reales de Ciencia de Datos, las funciones suelen provenir de simuladores computacionales complejos de caja negra (*black-box*), datos experimentales discretos o modelos donde la derivada analítica es intratable o costosa. Además, como estudiamos en la **Semana 4** del curso, aproximar $g'(x)$ mediante diferencias finitas convencionales es propenso a amplificar errores de redondeo por la precisión finita de la máquina ($\varepsilon_M \approx 2.22 \times 10^{-16}$). Por ello, el artículo de **Jerezano, Chicharro y Garrido-Sáez (2026)** propone un esquema bifásico que alcanza **orden óptimo 4 sin calcular ninguna derivada analítica**.

1.2. Estructura del Esquema Iterativo Propuesto

Los autores introducen un esquema predictor-corrector de dos pasos inspirado en Traub-Steffensen:

$$w_k = x_k + \beta[g(x_k)]^2, \quad y_k = x_k - m \frac{g(x_k)}{g[w_k, x_k]}, \quad x_{k+1} = x_k - m H(t_k) \frac{g(x_k)}{g[w_k, x_k]}, \quad (2)$$

donde $\beta \in \mathbb{C} \setminus \{0\}$, $g[w_k, x_k] = \frac{g(w_k) - g(x_k)}{w_k - x_k}$ es la diferencia dividida de primer orden, y la variable adimensional de control t_k se calcula mediante la rama principal del logaritmo complejo:

$$t_k = \left(\frac{g(y_k)}{g(x_k)} \right)^{1/m} = \exp \left(\frac{1}{m} \operatorname{Log} \left(\frac{g(y_k)}{g(x_k)} \right) \right). \quad (3)$$

Funcionamiento intuitivo paso a paso:

1. **Punto auxiliar adaptativo w_k :** Para trazar una recta secante sin derivadas, se perturba x_k sumándole $\beta[g(x_k)]^2$. Como $g(x_k) \rightarrow 0$ al acercarse a la raíz, $[g(x_k)]^2$ se desvanece cuadráticamente. Esto hace que w_k converja hacia x_k de forma autoadaptativa, eliminando la necesidad de calibrar un paso fijo h .
2. **Aproximación por diferencia dividida $g[w_k, x_k]$:** Se calcula la pendiente secante entre x_k y w_k . Por el Teorema del Valor Medio, aproxima fielmente a $g'(x_k)$ sin requerir diferenciación analítica.
3. **Paso intermedio predictor y_k :** Proporciona una estimación intermedia equivalente a un paso de Steffensen modificado.
4. **Paso corrector refinado con función de peso $H(t_k)$:** La variable t_k cuantifica la fracción de reducción del residuo lograda en el paso previo ($g(y_k) \ll g(x_k) \implies t_k \rightarrow 0$). La función $H(t_k)$ actúa como un **acelerador inteligente**: pondera el residuo y amplifica el avance final para alcanzar un orden de convergencia cuatro.

1.3. Condiciones Analíticas y Optimalidad de Kung-Traub

Mediante expansiones en series de Taylor del error $e_k = x_k - \alpha$, los autores imponen la anulación simultánea de los coeficientes de error de orden 1, 2 y 3, deduciendo cuatro condiciones que debe cumplir la función de peso $H(t)$ en el origen:

$$H(0) = 1, \quad H'(0) = 1, \quad H''(0) = 4, \quad |H'''(0)| < \infty. \quad (4)$$

Bajo estas condiciones, la ecuación asintótica del error toma la forma:

$$e_{k+1} = \frac{-H'''(0) + 3(9+m)B_1^3 - 6mB_1B_2}{6m^3} e_k^4 + \mathcal{O}(e_k^5), \quad (5)$$

demostrando que el método alcanza formalmente un **orden de convergencia cuatro** ($p = 4$).

Optimalidad según la conjetura de Kung-Traub (1974): Kung y Traub establecieron que el orden de convergencia p de cualquier método iterativo sin memoria que realice d evaluaciones funcionales por ciclo está acotado por $p \leq 2^{d-1}$. El esquema propuesto requiere exactamente $d = 3$ evaluaciones funcionales ($g(x_k), g(w_k), g(y_k)$) y cero derivadas. Por lo tanto, el orden máximo alcanzable es $p_{\max} = 2^{3-1} = 4$. Al alcanzar exactamente dicho límite con el mínimo número de evaluaciones, la familia es **óptima**. Prácticamente, el orden 4 implica que el número de cifras decimales exactas se cuadruplica en cada iteración ($10^{-2} \rightarrow 10^{-8} \rightarrow 10^{-32}$).

1.4. Familias Concretas Analizadas y Dinámica Compleja

Los autores seleccionan dos funciones de peso que satisfacen las condiciones del teorema:

- **Familia M_1 (Acelerador Polinomial):** $H_1(t) = 1 + t + 2t^2$.
- **Familia M_2 (Acelerador Racional):** $H_2(t) = \frac{1-t}{1-2t}$.

Estabilidad dinámica en el plano complejo: Al aplicar el operador de iteración racional $R(z, \beta)$ sobre el polinomio cuadrático $p(z) = (z-1)^2$, se demuestra que la raíz $\alpha = 1$ es un **punto superatractor** ($|R'(1, \beta)| = 0$). El parámetro β es crucial: valores moderados como $\beta = 0.01$ generan cuencas de atracción casi globales, evitando comportamientos caóticos. Además, la familia racional M_2 no presenta puntos fijos extraños atractores, lo cual garantiza una robustez superior frente a raíces espurias.

1.5. Limitaciones Identificadas

1. **Conocimiento previo de la multiplicidad m :** El algoritmo presupone conocida la multiplicidad exacta. Si m es incorrecto, el orden decae drásticamente a convergencia lineal o el método diverge.
2. **Sensibilidad paramétrica ante β y x_0 :** Valores grandes de β con semillas alejadas pueden inducir oscilaciones cíclicas o bifurcaciones.
3. **Manejo de ramas en potencias fraccionarias:** La evaluación de t_k en el plano complejo requiere fijar la rama principal del logaritmo para evitar saltos de discontinuidad.

2. Formulación del Problema Computacional

2.1. Contexto de Aplicación en Ciencia de Datos y Justificación

En la Ciencia de Datos y el aprendizaje automático, resolver ecuaciones no lineales es esencial para optimizar funciones de pérdida complejas (como regresión logística regularizada, estimación de máxima verosimilitud en distribuciones bimodales o ajuste no lineal de hiperparámetros). Frecuentemente, el cálculo simbólico de gradientes y matrices hessianas es inviable debido al tamaño masivo de los conjuntos de datos o la naturaleza opaca de las funciones objetivo. Disponer de familias de orden 4 libres de derivadas ofrece una alternativa computacional altamente eficiente para encontrar óptimos y ceros con rapidez sin incurrir en costos analíticos de diferenciación.

2.2. Objetivos del Proyecto

- **Objetivo General:** Desarrollar, validar y comparar en Python una suite computacional modular y reproducible que implemente las familias óptimas libres de derivadas M_1 y M_2 , reproduciendo con alta precisión los resultados del artículo base y contrastando su desempeño frente a métodos clásicos y de la literatura.
- **Objetivos Específicos:**
 1. Programar desde sus fundamentos matemáticos los métodos propuestos (M_1, M_2) y los métodos de contraste: Newton Modificado, Steffensen Modificado, SS, BH, KK y BB.
 2. Reproducir fielmente las pruebas del artículo en problemas con $m = 2, 3, 4$, empleando aritmética de precisión múltiple (`mpmath`) para alcanzar cotas de tolerancia de hasta 10^{-200} .
 3. Evaluar cuantitativamente la convergencia empírica mediante el residuo $|g(x_{k+1})|$, paso entre iteraciones $|x_{k+1} - x_k|$, orden computacional ACOC, índice de eficiencia (EI) y tiempo de CPU.
 4. Evaluar computacionalmente la estabilidad y sensibilidad ante variaciones de la semilla x_0 y el factor β , construyendo planos dinámicos en el plano complejo mediante `NumPy` y `Matplotlib`.

2.3. Preguntas de Investigación y Resultados Esperados

El proyecto responderá: ¿Compensa la eliminación de derivadas analíticas el costo de evaluar la diferencia dividida y la función de peso racional? y ¿Ofrece el acelerador racional M_2 mayor estabilidad numérica que el acelerador polinomial M_1 ante multiplicidades elevadas? Se espera constatar que el orden empírico alcance efectivamente $\text{ACOC} \approx 4.0000$ y que los residuos decrezcan por debajo de 10^{-200} en menos de 8 iteraciones.

3. Obtención de la Información y Articulación Curricular

3.1. Funciones Matemáticas de Prueba del Artículo

Para validar empíricamente la formulación computacional, se seleccionaron los tres problemas descritos en el artículo:

1. Problema 1: Ecuación de Estado de Van der Waals (Termodinámica):

$$g_1(x) = x^3 - 5.22x^2 + 9.0825x - 5.2675 = (x - 1.75)^2(x - 1.72). \quad (6)$$

Modela el comportamiento de gases reales. Presenta una raíz doble ($m = 2$) en $\alpha_1 = 1.75$ y una raíz simple inmediata en $\alpha_2 = 1.72$. La escasa separación entre raíces ($\Delta\alpha = 0.03$) evalúa la capacidad del algoritmo para converger a la raíz doble sin saltar a la raíz simple vecina.

2. Problema 2: Ley de Radiación de Planck en Cuerpo Negro (Física Térmica):

$$g_2(x) = \left(e^{-x} - 1 + \frac{x}{5}\right)^3. \quad (7)$$

Determina la longitud de onda de máxima densidad espectral de radiación. Posee una raíz triple ($m = 3$) en $\alpha \approx 4.96511423174428$. Al ser una función trascendente que combina términos exponenciales y lineales, examina el desempeño en funciones no algebraicas.

3. Problema 3: Polinomio Característico y Valores Propios (Álgebra Lineal):

$$g_3(x) = x^9 - 29x^8 + 349x^7 - 2261x^6 + 8455x^5 - 17663x^4 + 15927x^3 + 6993x^2 - 24732x + 12960. \quad (8)$$

Ecuación característica $\det(A - \lambda I) = 0$ de una matriz 9×9 , factorizable analíticamente como $(x - 3)^4(x - 8)(x - 5)(x - 4)(x - 1)(x + 1)$. Posee una raíz cuádruple ($m = 4$) en $\alpha = 3$ y cinco raíces simples. Sus coeficientes de gran escala evalúan la acumulación de errores de redondeo.

Cuadro 1: Caracterización de las funciones de prueba y parámetros experimentales.

Función de Prueba $g(x)$	Dominio Físico	Raíz α	Mult. (m)	Semillas (x_0)	Desafío Central	Número
$g_1(x) = (x - 1.75)^2(x - 1.72)$	Termodinámica	1.75	2	1.9, 3.0	Cercanía extrema a raíz simple ($\Delta = 0.03$).	
$g_2(x) = (e^{-x} - 1 + x/5)^3$	Radiación Térmica	≈ 4.96511	3	5.3, 10.0	Naturaleza trascendente exponencial.	
$g_3(x) = (x - 3)^4 \prod (x - \lambda_i)$	Álgebra Lineal	3.0	4	2.8, 3.5	Multiplicidad 4 y coeficientes de gran escala.	

3.2. Articulación con los Contenidos del Curso (Semanas 1 a 10)

El proyecto integrador conecta de forma directa con los temas del microcurrículo desarrollados hasta la fecha:

- **Semanas 2 y 3 (Aritmética IEEE-754, Épsilon y Cancelación Numérica):** En precisión estándar de 64 bits (`float64`), el ϵ de máquina es $\epsilon_M \approx 2.22 \times 10^{-16}$. En una raíz con multiplicidad $m = 4$, cuando el error absoluto es de apenas $e_k \approx 10^{-5}$, el residuo evaluado cae a $g(x_k) \approx 10^{-20}$, desbordando la precisión del estándar. Esto genera una **pérdida total de cifras significativas y división por ceros falsos**. Para evitar este fenómeno de cancelación catastrófica y poder reproducir las tablas del artículo hasta una tolerancia de $|g(x_{k+1})| < 10^{-200}$, justificamos el empleo de aritmética de precisión arbitraria (`mpmath` configurada con 4000 bits).
- **Semana 4 (Derivación Numérica y Diferencias Divididas):** El reemplazo de la derivada por la diferencia dividida $g[w_k, x_k]$ aplica el balance entre error de truncamiento y error de redondeo analizado en clase, utilizando la perturbación cuadrática $\beta[g(x)]^2$ como un paso adaptativo.
- **Semanas 6 a 9 (Álgebra Lineal Numérica y Valores Propios):** El Problema 3 vincula la solución de raíces con los autovalores de matrices en Ciencia de Datos (análisis de componentes principales, PCA), evidenciando el mal condicionamiento que inducen los autovalores múltiples.
- **Semana 10 (Solución de Ecuaciones No Lineales):** Las familias M_1 y M_2 extienden los métodos abiertos clásicos (Newton, Secante y Newton para sistemas), introduciendo métodos de alto orden de frontera científica contemporánea (2026).

4. Plan de Implementación y Criterios de Evaluación

4.1. Métodos a Programar y Métricas Cuantitativas

Para una contrastación rigurosa, se implementarán en Python los siguientes métodos:

- **Familias del Artículo Base:** Métodos libres de derivadas M_1 (polinomial) y M_2 (racional) con $\beta = 0.01$.
- **Métodos de Contraste con Derivadas:** Newton Modificado ($p = 2$), Método SS de Sharma-Sharma ($p = 4$) y Método BH de Behl et al. ($p = 4$).
- **Métodos de Contraste Libres de Derivadas:** Steffensen Modificado ($p = 2$), Método KK de Kumar et al. ($p = 4$) y Método BB de Behl et al. ($p = 4$).

Métricas Cuantitativas de Evaluación:

1. **Residuo y Precisión:** Residuo $|g(x_{k+1})|$ e incremento entre pasos $|x_{k+1} - x_k|$. Criterio de parada: $|x_{k+1} - x_k| + |g(x_{k+1})| < 10^{-200}$.
2. **Orden Computacional de Convergencia Aproximado (ACOC):** Fórmula de Cordero y Torregrosa:
$$\text{ACOC} \approx \frac{\ln |(x_{k+1} - x_k)/(x_k - x_{k-1})|}{\ln |(x_k - x_{k-1})/(x_{k-1} - x_{k-2})|}, \quad \text{con valor objetivo} \approx 4.0000. \quad (9)$$
3. **Índice de Eficiencia de Kung-Traub ($EI = p^{1/d}$):** Mientras Newton modificado ($p = 2, d = 2$) tiene $EI \approx 1.4142$, las familias propuestas M_1 y M_2 ($p = 4, d = 3$) alcanzan $EI = 4^{1/3} \approx 1.5874$, entregando mayor rendimiento por evaluación computacional.
4. **Tiempo de CPU:** Tiempo de ejecución en milisegundos mediante la librería estándar `time`, promediando 50 ejecuciones.

4.2. Arquitectura Modular del Software y Reproducibilidad

El código se alojará en el repositorio <https://github.com/ailynsof/Proyecto-M-todos-Numericos> con la siguiente estructura modular en `src/`:

- `src/methods.py`: Implementación de los algoritmos M_1, M_2 , Newton modificado, Steffensen, SS, BH, KK y BB.
- `src/benchmarks.py`: Funciones g_1, g_2, g_3 , multiplicidades, derivadas analíticas y semillas x_0 .
- `src/metrics.py`: Funciones de cálculo de ACOG, residuos, análisis de tiempos y exportador automático a tablas L^AT_EX.
- `src/dynamics.py`: Generador de planos dinámicos y mapas de atracción sobre mallas complejas con NumPy y Matplotlib.
- **Garantía de Reproducibilidad:** Se fijará una semilla aleatoria (`numpy.random.seed(42)`) y se proveerá un archivo `requirements.txt`.

4.3. Cronograma de Fases del Proyecto Integrador

- **Fase 1 (Semana 5 — Entrega Actual):** Comprensión del artículo científico, formulación matemática y estructuración del plan.
- **Fase 2 (Semana 10):** Implementación computacional completa en Python, contrastación de métodos y reproducción de Tablas 1, 2 y 3.
- **Fase 3 (Semana 16):** Análisis de estabilidad dinámica, conclusiones globales, entrega de informe final consolidado y sustentación oral.

Declaración de Uso de Herramientas de Inteligencia Artificial

En estricto cumplimiento del Reglamento de Integridad Académica de la Universidad Externado de Colombia y las normas del proyecto integrador, los integrantes del equipo declaramos que empleamos herramientas de Inteligencia Artificial Generativa exclusivamente como apoyo técnico para la diagramación tipográfica en L^AT_EX y la revisión de estilo. Los autores asumimos la autoría total, el entendimiento matemático y la responsabilidad académica sobre el contenido, formulaciones y análisis expuestos en este informe.

Referencias

- [1] Jerezano, J. H., Chicharro, F. I., & Garrido-Sáez, N. (2026). Derivative-free optimal multiple-root finding family of iterative methods. *Computational and Applied Mathematics*, 45:304.
- [2] Kung, H. T., & Traub, J. F. (1974). Optimal order of one-point and multipoint iteration. *Journal of the ACM*, 21(4), 643–651.
- [3] Cordero, A., & Torregrosa, J. R. (2007). Variants of Newton's method using fifth-order quadrature formulas. *Applied Mathematics and Computation*, 190(1), 686–698.
- [4] Sharma, J. R., & Sharma, R. (2010). A new family of modified Ostrowski's methods with accelerated eighth-order convergence. *Numerical Algorithms*, 54(2), 243–258.
- [5] Behl, R., Cordero, A., Motsa, S. S., & Torregrosa, J. R. (2016). An optimal fourth-order family of methods for multiple roots and its dynamics. *Journal of Computational and Applied Mathematics*, 301, 169–178.
- [6] Kumar, S., Kumar, D., Sharma, J. R., & Jäntschi, L. (2020). An optimal fourth-order derivative-free family of iterative methods for multiple roots. *Mathematics*, 8(8), 1275.
- [7] Turner, P. R., Arildsen, T., & Kavanagh, K. (2018). *Applied Scientific Computing: With Python*. Springer.